

Superpowers vs. Kit vs. Dynamic Workflow — Three-Arm Bake-Off

Superpowers vs. Kit vs. Dynamic Workflow

A controlled, blind, three-arm bake-off on one real UE5/GAS task.

TL;DR

The newly-installed `superpowers` plugin (a process-methodology framework) was put head-to-head against Max's ClaudeKit specialists and a dynamic Workflow, all implementing the **same** `UTOGGA_ChainedAttack` combo ability in TOG-Remake, judged blind by a 5-lens panel.

- 🏆 **Highest quality: the dynamic Workflow (Arm C) — 80/100 — but at ~3.2x the kit's token cost.**
- 💰 **Most efficient: the Kit (Arm B) — 66/100 at ~174k tokens / ~12 min — best quality-per-token, and it Pareto-dominates superpowers** (higher quality *and* cheaper).
- 🔴 **Superpowers (Arm A) came last — 60/100 at ~210k tokens — and shipped a critical activation bug** its TDD/verification discipline couldn't catch, because the project can't compile on this machine.

Bottom line for the original question ("can superpowers compete with parts of the kit?"): On this task, no — superpowers was dominated by the kit on both axes. But this is **n=1** (one task, one run per arm, one judge per lens, one model). Treat as *directional*, not a verdict on the framework.

Setup & fairness controls

Control	What was done
Task	Implement <code>UTOGGA_ChainedAttack</code> (persistent multi-step combo GameplayAbility), fully specified in <code>docs/in-depth/TOG-Combo-System-Architecture/ §4-10 + AgentRef</code> .
Same starting state	Three git worktrees off <code>master @ 679157e</code> , isolated so arms never collide.
Same model	All implementation + judging pinned to sonnet — tests <i>method</i> , not model.
Equal review rigor	Each arm got a real implement → review → fix loop (Arm B's was added after a mid-run catch — see Process Notes).
Blind judging	Diffs relabeled Submission 1/2/3; arm↔submission mapping withheld from judges; verified no identity leaks in the diffs.

Control	What was done
Workflow compliance	Each arm's transcript audited against its planned workflow before judging (all three: faithful).
Static judging	TOG-Remake is not buildable on this Mac and has no test harness — judged by reading the diff vs. the spec. Authored tests counted as artifacts, not run. This was declared up front.

The three arms (all same task): - **Arm A — Superpowers methodology.** One agent driving the full skill spine: brainstorming → writing-plans → test-driven-development → systematic-debugging → requesting/receiving-code-review → finishing-a-development-branch . - **Arm B — Kit specialists.** game-developer implements → code-reviewer fan-out → game-developer applies fixes. No superpowers (structurally — those agents have no Skill tool). - **Arm C — Dynamic Workflow.** Decompose-and-fan-out: 2 parallel spec/dependency readers → implementer → 3 adversarial verify lenses → fixer (7 agents).

Quality scores (blind 5-lens panel, 0–100)

Lens	Arm C — Workflow	Arm A — Superpowers	Arm B — Kit
GAS / architecture fidelity	82	48	65
C++ / UE5 correctness	72	44	63
Completeness vs spec	82	76	58
Code quality & hygiene	82	78	72
No-fabrication / conventions	82	55	73
Average	80	60	66

(Submission mapping, held private from judges during scoring: Sub-1 = Arm C, Sub-2 = Arm A, Sub-3 = Arm B.)

Efficiency (implement + review + fix totals)

Arm	Quality	Output tokens	Wall-clock	Agents	Quality / 100k tok
B — Kit	66	~174k	~12 min	3	37.9
A — Superpowers	60	~210k	~23 min	2	28.6
C — Workflow	80	~558k	~27 min	7	14.3

(Token totals: Arm A 209,775; Arm B $89,612 + 58,134 + 26,577 = 174,323$; Arm C 558,253. Panel judging (~470k) and compliance audits (~107k) are orchestration overhead, not charged to any arm.)

Key findings

1. Superpowers shipped a critical bug its own process should have caught

The panel flagged a **critical first-step defect** in Arm A: `AdvanceIndex()` early-returns when `ActiveChain` is null, but `ActiveChain` is null on the first call — so `AttackIndex` stays at `-1`, `SelectMontage()` returns `nullptr`, and **the ability aborts on activation; the chain never runs**. Arm A ran TDD *and* a forked code-review *and* systematic-debugging — and still missed it. Why: the bug is a runtime activation-order issue that authored-but-unrun tests can't surface. **The static environment neutralized superpowers' signature disciplines.**

2. The kit's review loop caught the exact class of bug superpowers missed

Arm B's `code-reviewer` pass explicitly flagged the step-ordering hazard, and the fix specifically guarded the first-step case (`SelectChain()` first when `ActiveChain == nullptr`). The kit's specialist+review loop dodged the bug superpowers fell into — at lower cost.

3. The workflow won on quality by being broadest *and* most adversarial

Arm C's 3-lens adversarial verify caught 8 findings (2 critical) and its fixer addressed them; it also closed the `State.Attacking` tag gap and wired `MeleeAttack`. That breadth earned top fidelity/completeness — but cost 558k tokens (6× Arm B's implementation alone).

4. Scope discipline cut both ways

Arm B changed only the 2 target files — praised under **code quality / scope discipline** (72) but penalized under **fidelity/completeness** (65/58) for leaving the `State.Attacking` tag registration as a documented TODO. Arms A and C registered the tag (broader scope) and scored higher on fidelity. *There's no free lunch: the minimal-diff that reviewers love is the same diff that "completeness" judges dock.*

Threats to validity (read before acting)

- **n = 1.** One task, one run per arm, one judge per lens, one model. The critical bug that sank Arm A could be run-to-run variance. **Do not promote any single arm to policy on this alone.**
- **Static judging favors the cautious.** No compile/run means correctness was a judge's *read*, not execution. This structurally penalized the TDD-heavy arm (A) and rewarded breadth (C).

- **Single judge per lens** (not a per-lens panel) — less robust than the plan's ideal; scores are indicative.
- **Arm B's review loop was added mid-run** after a fairness catch (see below); its tokens include that, fairly.

Recommendation: superpowers posture

The data supports the **"keep, tame the hook"** posture from planning — now with evidence:

1. **Don't make superpowers the default spine.** Its full ceremony cost more than the kit and produced lower-quality code *here*, largely because its TDD/verify edge is blunted whenever a project can't run tests (true for TOG-Remake, and any UE5-on-Mac work).
2. **Keep its genuinely-additive skills** — brainstorming , writing-plans , receiving-code-review — which the kit lacks. Reach for them deliberately, not via the always-on enforcement hook.
3. **Default to the kit for efficiency-sensitive work**, and **escalate to a dynamic Workflow when correctness matters more than budget** (it bought the best quality, at a price).
4. **Re-run before committing to policy.** Suggested next pass: 2–3 tasks × 2 seeds, and at least one task in a *buildable* repo so superpowers' TDD discipline can actually execute — that's the fair rematch.

Process notes (honesty log)

- **Arm B initially skipped its review loop.** The first run dispatched only the game-developer implementer; the planned code-reviewer fan-out + revise was missed. Caught mid-run (user flagged the agent count), then run properly so all three arms had equal review rigor. Without this, the kit would have been unfairly handicapped in its own bake-off.
- **Blind-leak fix.** Arm A's diff originally included a docs/superpowers/plans/... path — an identity giveaway. All diffs were renormalized to Source/ changes only before judging.
- **Workflow compliance verified from transcripts**, not self-reports: Arm A invoked all 7 superpowers skills in order with a real forked reviewer; Arm C ran all 4 phases / 7 agents with zero superpowers leakage; Arm B's implement→review→fix chain confirmed by git log + structural no-Skill-tool guarantee.

Appendix — artifacts

Item	Location
Arm A commit (superpowers)	bakeoff-arm-a @ 598b76e
Arm B commits (kit)	bakeoff-arm-b @ 928240e → 43186e4

Item	Location
Arm C commit (workflow)	bakeoff-arm-c @ 38f7cd6
Blind diffs	/tmp/bakeoff/submission-{1,2,3}.diff
Judge panel transcript	workflow wf_bc14a7ce-52e
Arm C workflow transcript	workflow wf_52871b9b-b30